МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра АПУ

ОТЧЕТ

по практической работе №4

по дисциплине «Автоматизация тестирования»

Тема: Тестирование с помощью Selenium webdriver и Python

Студентка гр. 4370 _____ Александрова В.Р.

Преподаватель _____ Турнецкая Е.Л.

Санкт-Петербург

2026

Цель работы

Получение практических навыков по тестированию веб-элементов с помощью Selenium Webdriver и Python.

Задачи

1. Установить программное окружение проекта по автоматизированному тестированию.
2. Реализовать проект по запуску браузера под управлением Selenium и отобразить в нем страницу веб-системы.
3. Разработать проект по нахождению веб-элементов на учебном ресурсе с помощью программных методов Selenium.
4. Создать проект по заполнению полей формы регистрации с использованием явных и неявных ожиданий Selenium.
5. Зафиксировать результаты тестирования в отчете.

Ход работы

Часть 1

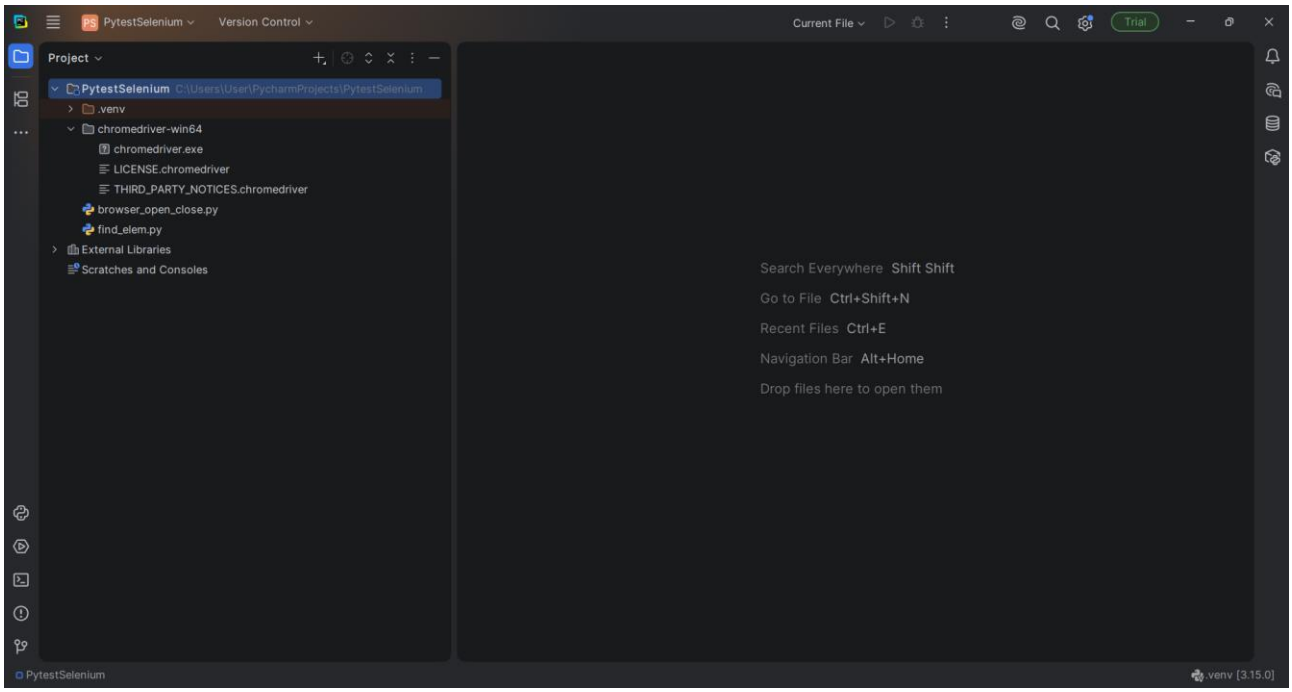


Рис. 1 — Структура проекта в IDE PyCharm с виртуальным окружением и ChromeDriver

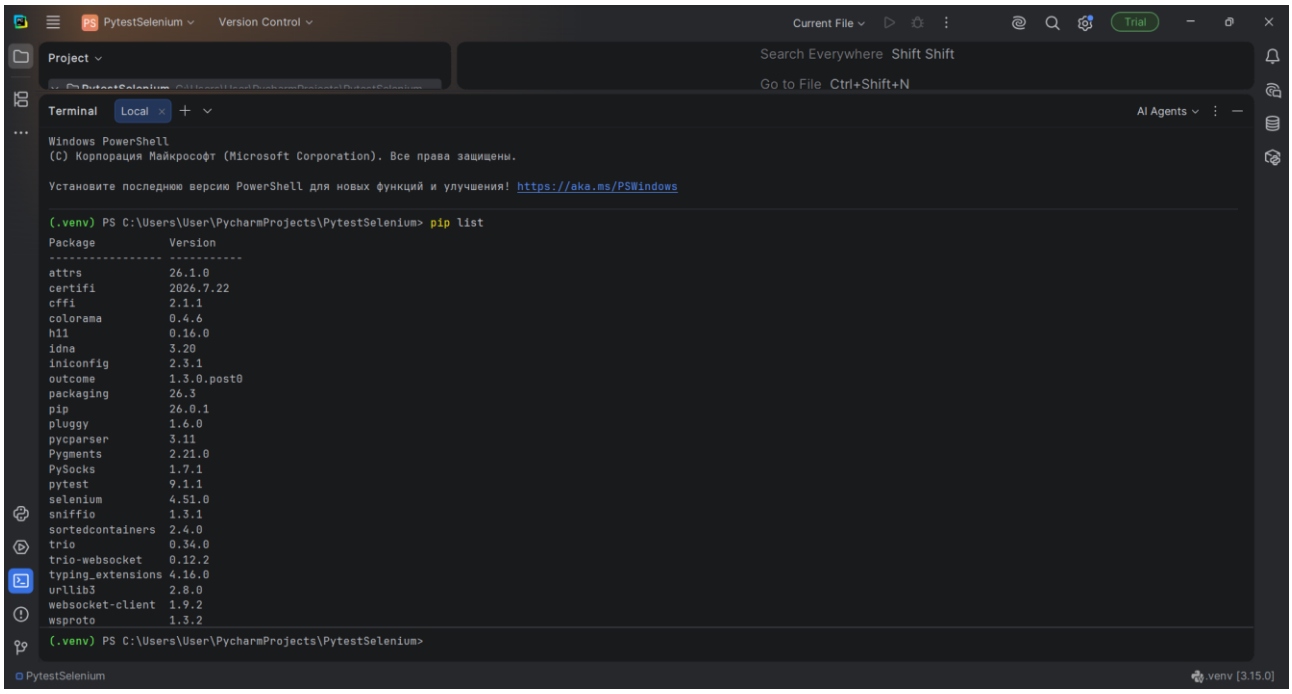


Рис. 2 — Список установленных пакетов в виртуальном окружении (.venv) с подтверждением установки selenium и pytest

Часть 2

Листинг программы:

```
from selenium import webdriver
import time
from selenium.webdriver.chrome.service import Service as ChromeService

# Путь к исполняемому файлу chromedriver.exe
chrome_driver_path = 'chromedriver-win64/chromedriver.exe'

# Создание сервиса Chrome
chrome_service = ChromeService(executable_path=chrome_driver_path)

# Создание экземпляра браузера
driver = webdriver.Chrome(service=chrome_service)

try:

 # Открытие веб-сайта
 driver.get('https://dead-god.ru')

 # Задержка в 5 секунд (явное ожидание)
 time.sleep(5)

 # Сообщение если ошибка
except Exception as e:
 print(f"Произошла ошибка: {e}")

finally:
 # Закрытие браузера
 driver.quit()
```

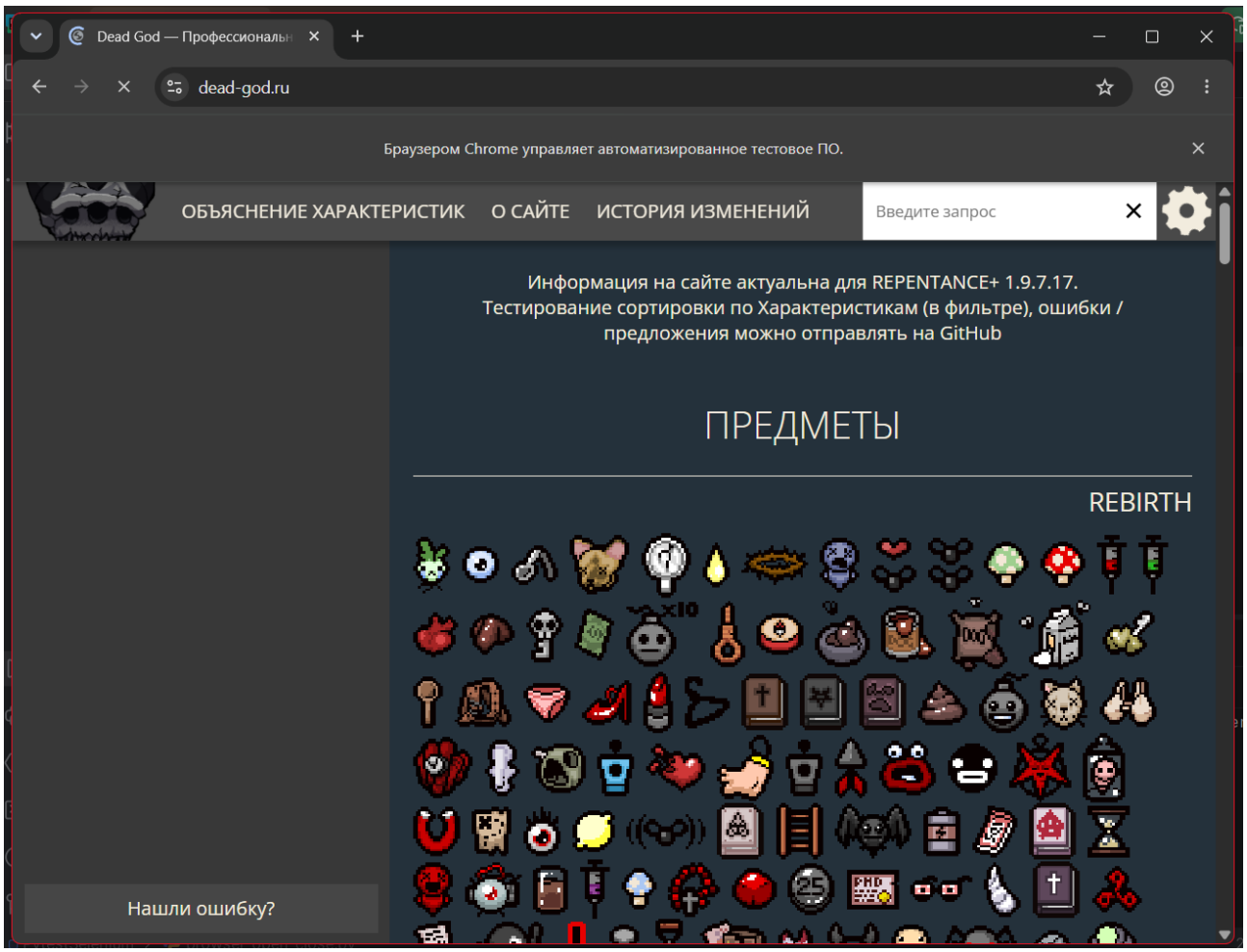


Рис. 3 — Окно браузера Chrome под управлением Selenium WebDriver

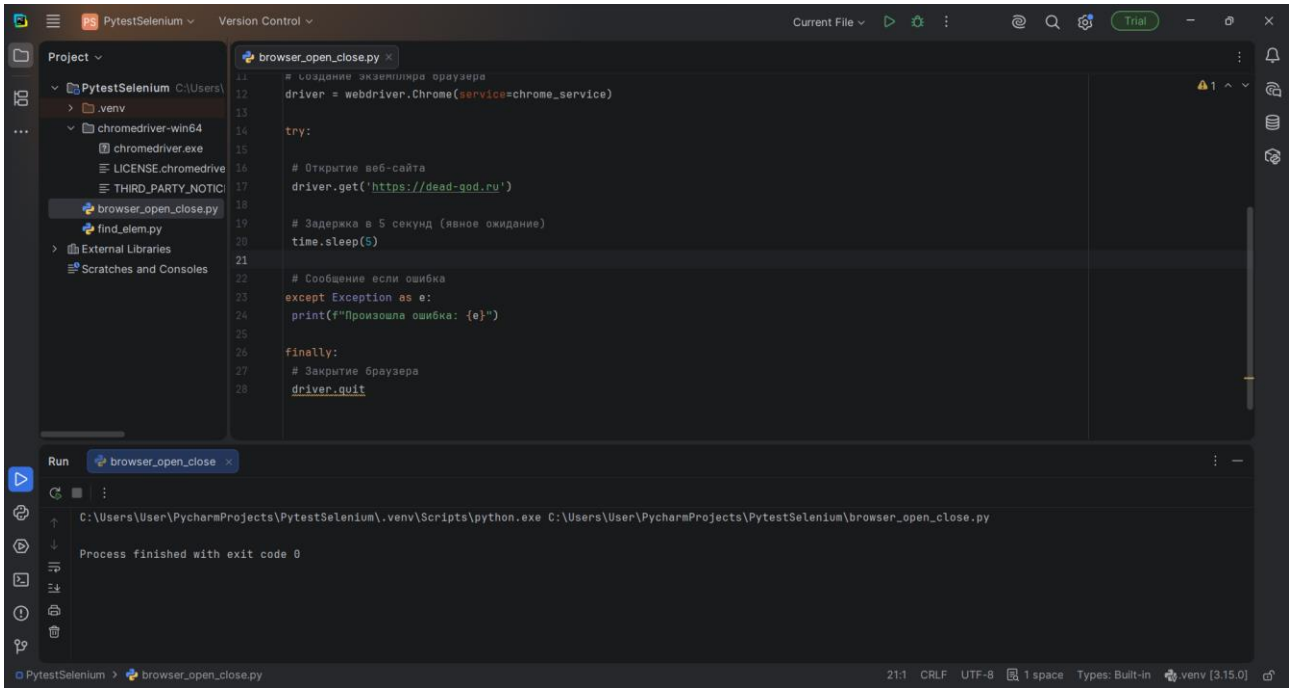


Рис. 4 — Успешное выполнение browser_open_close.py в IDE PyCharm

Часть 3

Листинг программы:

```
import time
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.common.by import By

# Путь к исполняемому файлу chromedriver.exe
chrome_driver_path = 'chromedriver-win64/chromedriver.exe'

# Создание сервиса Chrome
chrome_service = ChromeService(executable_path=chrome_driver_path)

# Создание экземпляра браузера
browser = webdriver.Chrome(service=chrome_service)

URL = 'https://qa-test-selectors.netlify.app/'
VARIANT = 1  # Ваш вариант

try:
    # Открытие веб-сайта
    browser.get(URL)
    browser.implicitly_wait(10)

    # Клик по кнопке варианта 1
    variant_button = browser.find_element(
        By.XPATH, f'//button[contains(@class, "variant__btn") and
text()="{VARIANT}"]'
    )
    variant_button.click()
    time.sleep(1)

    # Поиск заголовков h1 (должно быть 6 карточек)
    h1_elements = browser.find_elements(By.TAG_NAME, 'h1')
    print('Элементы с тегом h1:', len(h1_elements))
```

```
    # Поиск контейнеров карточек по классу imageContainer
    cards_by_class = browser.find_elements(By.CLASS_NAME, 'imageContainer')
    print('Элементы с классом imageContainer:', len(cards_by_class))

    # Поиск всех изображений img карточек
    img_elements = browser.find_elements(By.TAG_NAME, 'img')
    print('Элементы с тегом img:', len(img_elements))

    # Поиск по атрибуту data-type (выбираем селектором карточки на странице)
    elements_by_data_type = browser.find_elements(
        By.CSS_SELECTOR, '[data-type]'
    )
    print('Элементы с атрибутом data-type:', len(elements_by_data_type))

    # Поиск конкретного элемента по ID
    elements_by_id = browser.find_elements(By.ID, "sad")
    print("Элементов по ID:", len(elements_by_id))

finally:
 # Закрытие браузера
    browser.quit()
```

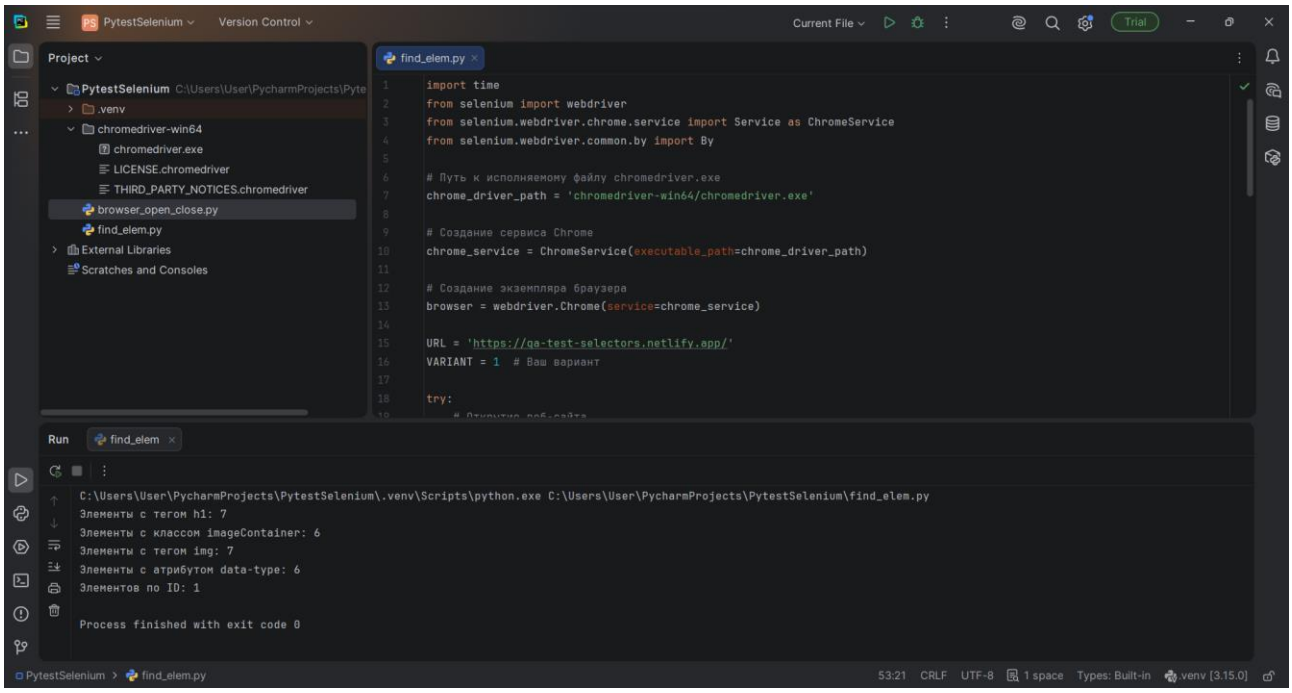


Рис. 5 — Результаты выполнения автотеста поиска элементов в консоли PyCharm

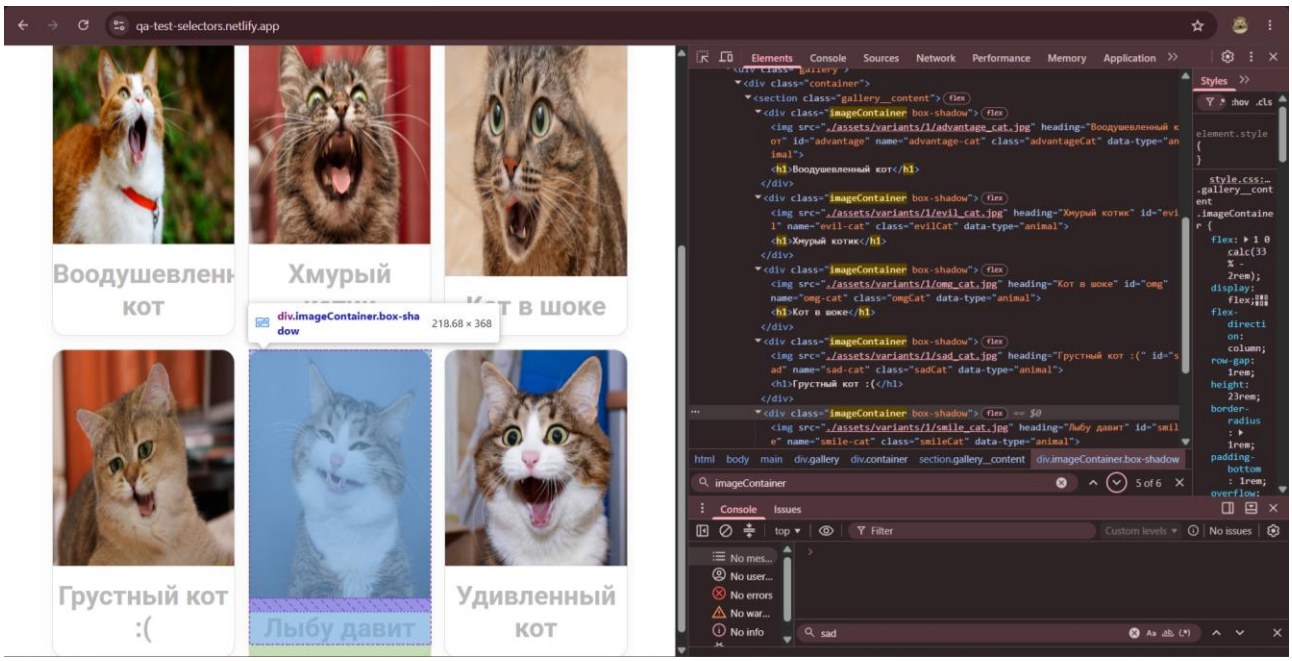


Рис. 6 — Ручная проверка карточек с классом imageContainer в Chrome DevTools (найдено 6 элементов)

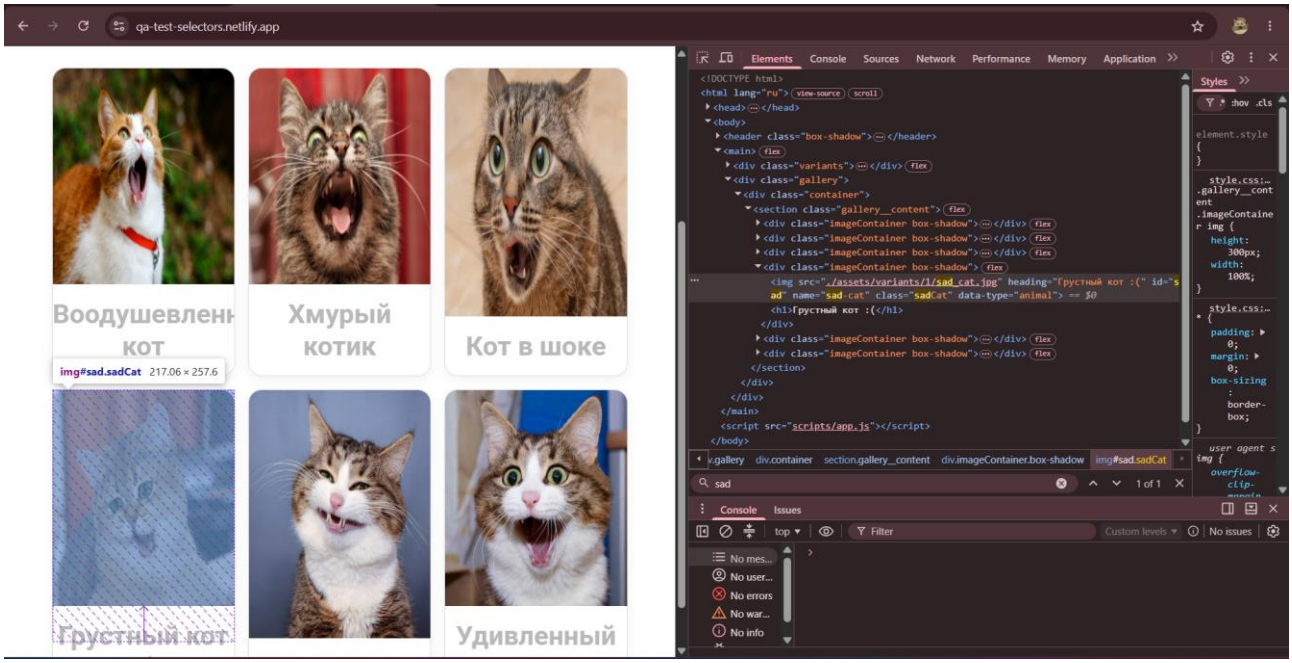


Рис. 7 — Ручная проверка уникального элемента с id = “sad”

Часть 4

Листинг программы:

```
import time
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.common.by import By

chrome_driver_path = 'chromedriver-win64/chromedriver.exe'
chrome_service = ChromeService(executable_path=chrome_driver_path)
browser = webdriver.Chrome(service=chrome_service)

URL = 'https://qa-course.netlify.app/registration-form-timer'

try:
    browser.get(URL)
    browser.implicitly_wait(4)

    # Поле "Имя" — по классу
    field_first_name = browser.find_element(By.CLASS_NAME, 'formControl')
    field_first_name.send_keys('Максим')

    # Поле "Фамилия" — по имени атрибута
    field_last_name = browser.find_element(By.NAME, 'lastName')
    field_last_name.send_keys('Смирнов')

    # Поле "Город" — по XPATH
    field_city = browser.find_element(By.XPATH, "//input[@name='city']")
    field_city.send_keys('Казань')

    # Поле "E-mail" — по CSS-селектору
    field_email = browser.find_element(
        By.CSS_SELECTOR, "input[type='email'], input[name='email']"
    )
    field_email.send_keys('smirnov_qa@etu.ru')
```

```
    # Кнопка "Отправить" — по тегу
    submit_button = browser.find_element(By.TAG_NAME, 'button')
    submit_button.click()

    print('Форма успешно заполнена и отправлена')
    time.sleep(5)

except Exception as e:
    print(f'Произошла ошибка: {e}')
finally:
    browser.quit()
```

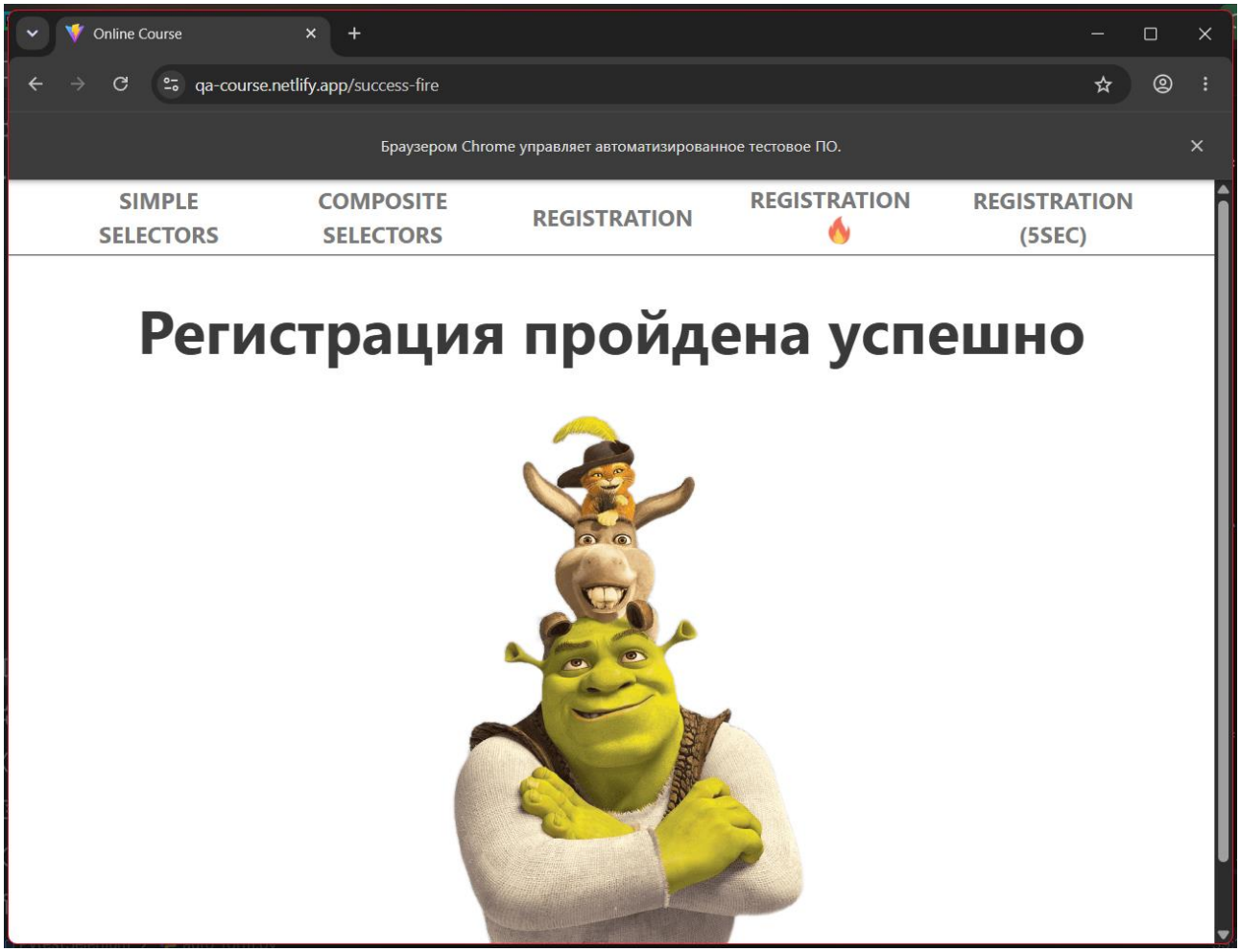


Рис. 8 — Страница учебного тренажера с сообщением «Регистрация пройдена успешно»

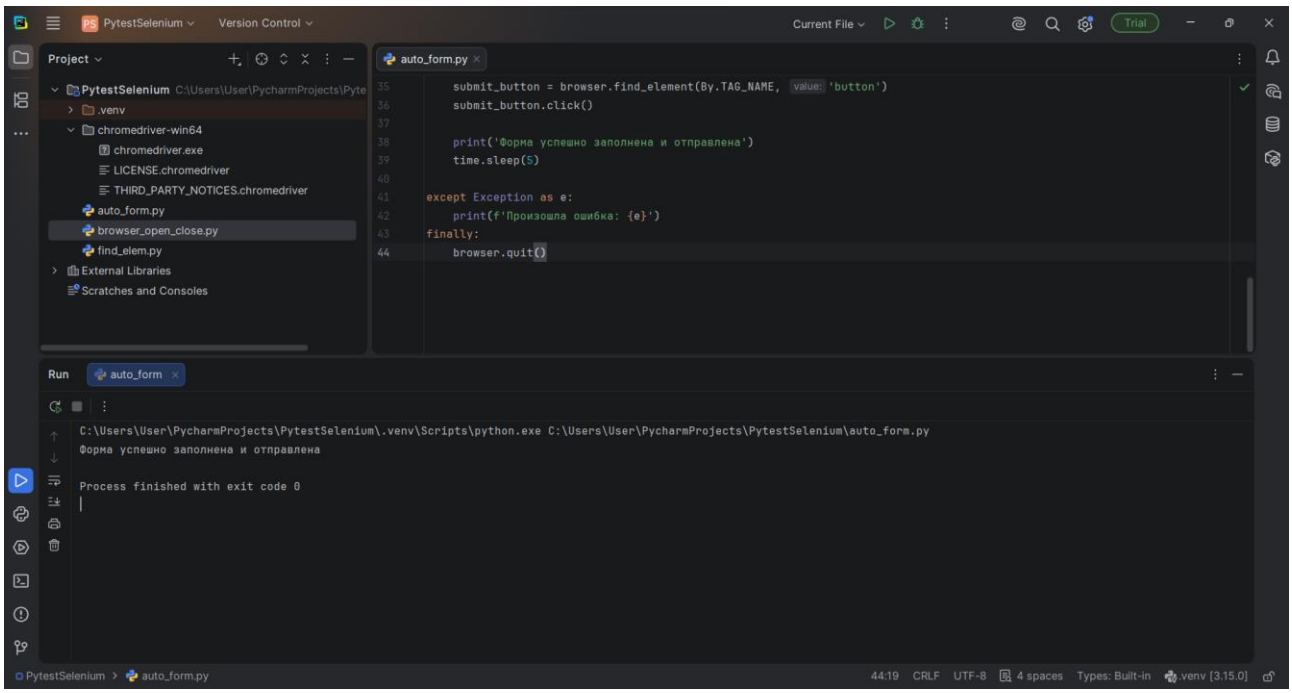


Рис. 9 — Лог успешного выполнения автотеста заполнения формы в PyCharm

Вывод

В ходе выполнения практической работы были получены практические навыки разработки и программной отладки сценариев автоматизированного тестирования веб-приложений с помощью библиотеки Selenium WebDriver и языка программирования Python в среде разработки PyCharm.

На первом этапе работы было настроено изолированное виртуальное окружение, подключен драйвер ChromeDriver и реализован базовый скрипт управления браузером Google Chrome с проверкой доступности веб-ресурса и корректным завершением сессии с помощью конструкции try-finally.

В основной части работы было проведено автоматизированное тестирование учебного тренажера по индивидуальному варианту №1 методом «Белого ящика». Были реализованы сценарии взаимодействия с веб-элементами: программный клик по кнопке варианта, поиск и расчет количества элементов через различные типы локаторов (By.TAG_NAME, By.CLASS_NAME, By.CSS_SELECTOR, By.ID), а также ручная верификация найденных селекторов через панель инструментов разработчика Chrome DevTools. Кроме того, был успешно реализован сценарий автоматического заполнения и отправки формы регистрации, ограниченной жестким таймером в 5 секунд.

В процессе выполнения работы возникли сложности, вызванные особенностями верстки веб-страницы. При поиске по общим тегам h1 и img автотест обнаружил по 7 элементов вместо 6 ожидаемых предметных карточек из-за наличия глобального заголовка сервиса и логотипа в шапке сайта. При автоматизации формы регистрации возникла проблема однотипности верстки: все поля ввода имели одинаковый класс formControl, что вызывало ошибку NoSuchElementException при попытке прямого поиска поля по специфичному классу, а лимит времени в 5 секунд делал невозможным использование статических задержек.

Данные проблемы были успешно решены путем подбора более точных составных локаторов (по классу контейнера .imageContainer и атрибуту [data-type]), грамотного распределения методов поиска (TAG_NAME, NAME, CLASS_NAME, XPATH, CSS_SELECTOR) без использования хрупких индексов, а также устранения избыточных пауз для мгновенного заполнения и отправки формы до блокировки интерфейса таймером.

Полученные навыки наглядно демонстрируют преимущество использования программного стека Python-Selenium перед ручным тестированием: скрипты позволяют с высокой скоростью взаимодействовать с интерфейсом, обходить динамические ограничения по времени и обеспечивают стабильное проведение регрессионного и приемочного (E2E) тестирования веб-систем.

Список использованных источников

1. Турнецкая, Е. Л. Тестирование с помощью Selenium webdriver и Python: практическое задание № 4 / Е. Л. Турнецкая. – Санкт-Петербург: СПбГЭТУ «ЛЭТИ», 2024. – 38 с.

2. Официальный портал проекта Selenium. – URL: https://www.selenium.dev/ (дата обращения: 14.10.2026).

3. Документация по Selenium WebDriver. – URL: https://www.selenium.dev/documentation/webdriver/ (дата обращения: 14.10.2026).

4. WebDriver: W3C Recommendation. – URL: https://w3c.github.io/webdriver/ (дата обращения: 14.10.2026).